

ripwire

The ripgrep of AI context.

A zero-runtime-dependency C++23 CLI that maps any codebase into a ranked, deterministic call graph for coding agents — and puts a tripwire on every claim it emits.

rip — the speed half

Structural answers in tens of milliseconds, from a call graph it built itself.

wire — the honesty half

Unprovable totals ship labelled as floors. A zero means none found — never none exists.

// the problem

Your agent reads the whole library to answer one question

Blind grep, then whole-file reads. Most of what enters the context window is never used — it is paid for anyway, on every step of every task.

And bigger context is not better context: irrelevant text actively degrades the answer. The job is selection — a small, ranked, structural map beats a pile of open files.

Speed caveat travels with the number: ripwire answers from a warm, pre-built index; competitor medians include their per-question work. Same instances, same gold, same metric code.

0.074 s

median answer, warm index — head-to-head, N = 60

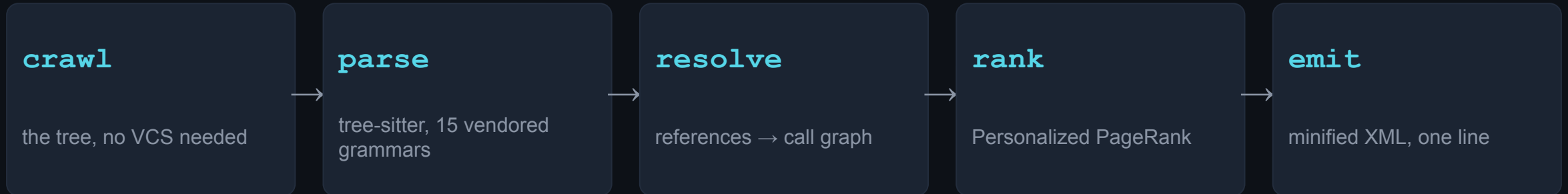
−39.4%

token ceiling vs the un-routed baseline — LocBench cost ledger, N = 243

// one binary, the whole pipeline

Crawl → parse → resolve → rank → emit.

Deterministic, end to end.



byte-identical

two runs, same bytes — a gate on every push, not a tendency; warm equals cold

zero runtime deps

CMake + a C++23 compiler; builds with the network off — vendored everything

11 languages

C/C++, ObjC, Python, TS/JS, Java, Go, Rust, Ruby, Swift, C#, Bash — plus JSON and Metal (a C++ dialect)

agent-native

an MCP server and 123 long flags behind one `--help` that is always the authority

// every feature, one screen

Seven families of questions it answers

understand cold	<i>What is this repo, and what matters in it?</i>	<code>--for --tree --lego --exemplar --recall --top-k --token-budget</code>
navigate	<i>Who calls this? Safe to change? Which tests?</i>	<code>--callers --callees --uses --impact --path --connect --affected --situ --test-gate --grep</code>
detail ladder	<i>Show me more — but only where it pays.</i>	<code>--detail --pack-signatures --outline --expand --compress</code>
quality & risk	<i>Where is the risk, and did I just add some?</i>	<code>--hotspots --clones --metrics --deps --lint --quality-delta --edit-check --pr-context --merge-scout</code>
self-diagnosis	<i>Is my setup actually working?</i>	<code>--doctor</code>
security	<i>Is this agent skill file safe to install?</i>	<code>--scan-skill --scan-skills</code>
knobs & modes	<i>Shape, format, cache, budget.</i>	<code>--json --format --mcp</code>

ripwire --help is generated from the binary's own flag table — 123 long flags; docs/COMMANDS.md documents 101 of them with real recorded output

```
// reach for it at the moment, not the manual
```

The reflexes: which verb fires when

Landing cold on a task

```
--for="<task>" · --pack-task
```

ranked, budgeted context in one call

Stack trace in hand

```
--from-trace
```

frames mapped to symbols, innermost body included

“Is it safe to change X?”

```
--impact · --uses
```

the whole blast radius, not just 1-hop callers

Just edited a symbol

```
--edit-check
```

did the contract change, who breaks — ~ms, warm

About to write a helper

```
--exemplar · --grep
```

imitate the house pattern; catch the duplicate early

Landing several branches

```
--merge-scout
```

pairwise conflict sites + a landing order

Before you call it done

```
--quality-delta · --test-gate
```

only what you made worse; exactly which tests to run

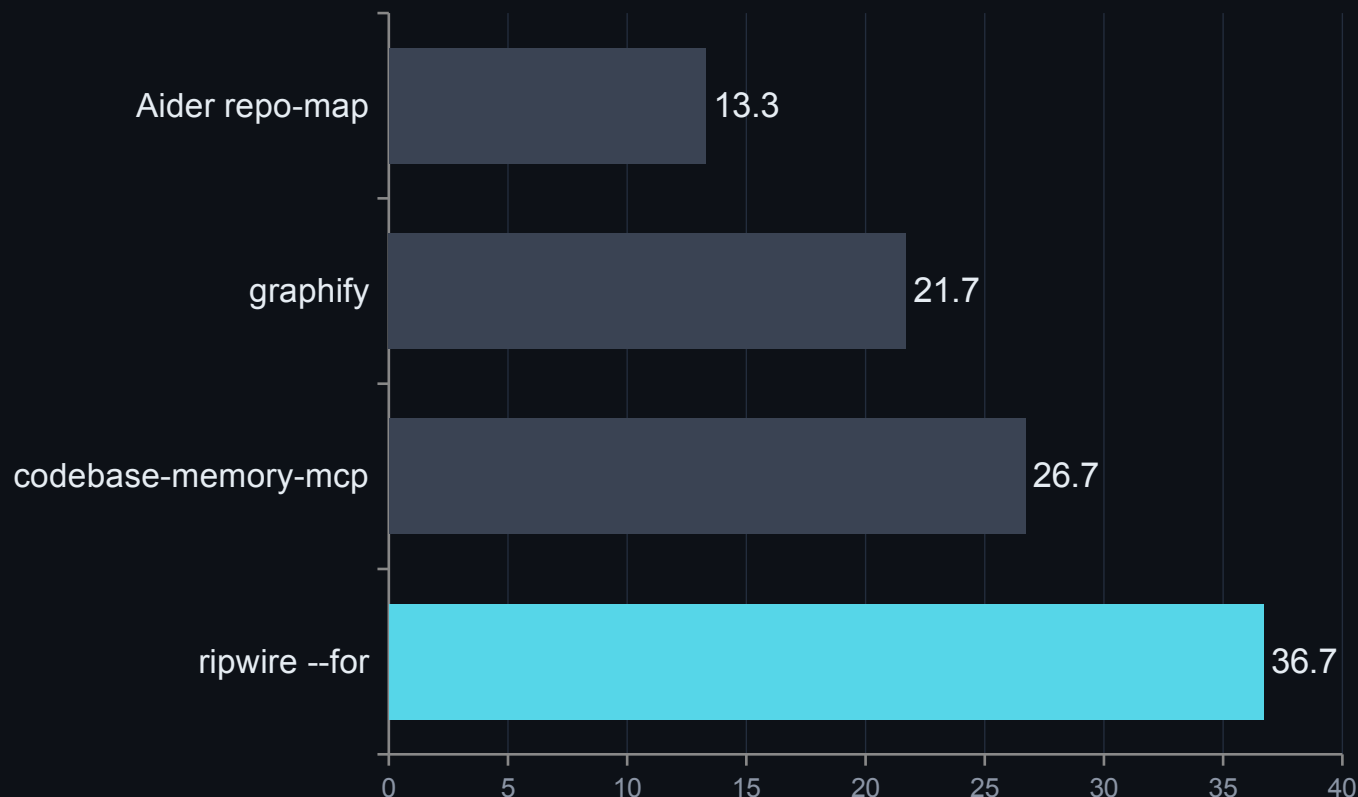
Reviewing a diff

```
--pr-context
```

per-file blast radius, tests-to-run, hotspot flags

// same instances, same gold, same metric code

Head-to-head: strict localization, four tools



36.7%

strict file@10 — every gold file in the top ten
N = 60 paired, zero exclusions

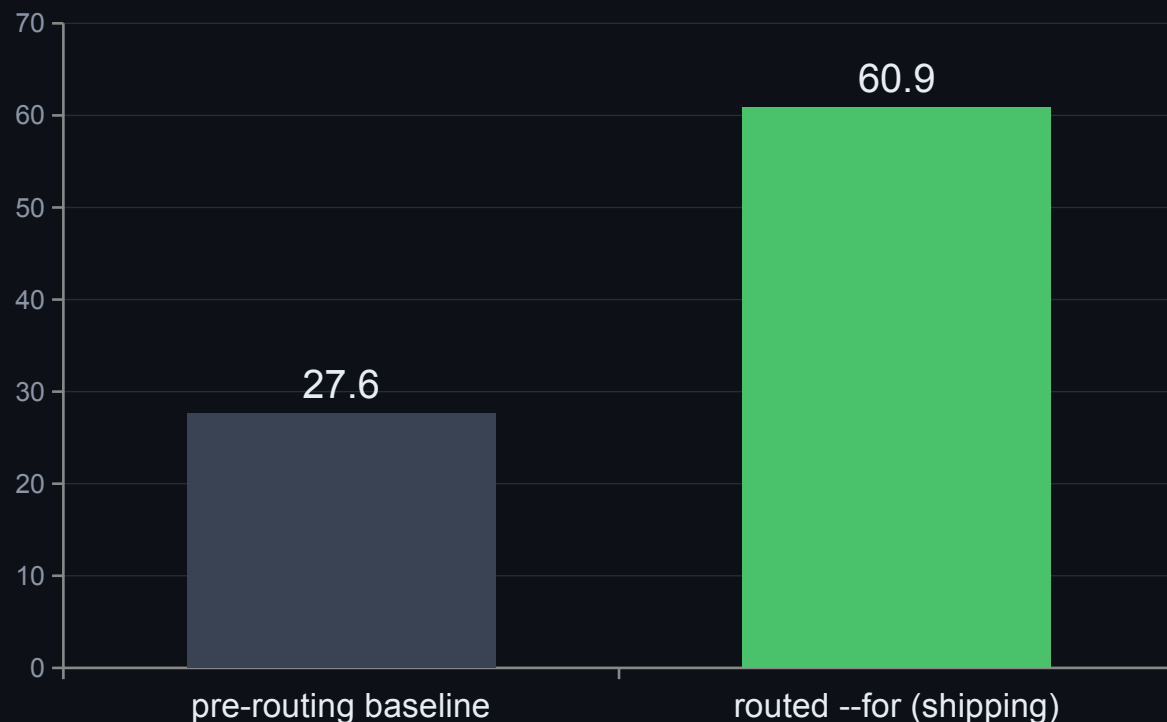
Median wall clock

ripwire 0.074 s · cmm 1.14 s · Aider 2.5 s · graphify 5.8 s

≈34× vs Aider is derived from those medians — and the caveat ships with it: ripwire is warm on a pre-built index.

// held-out, repo-disjoint, frozen dataset

Routing the ranker: +33 points, measured properly



+33.33pp

paired delta, 243 held-out instances in 78 repository clusters

+25.00pp

clustered-bootstrap 95% lower bound — the honest floor

-39.4%

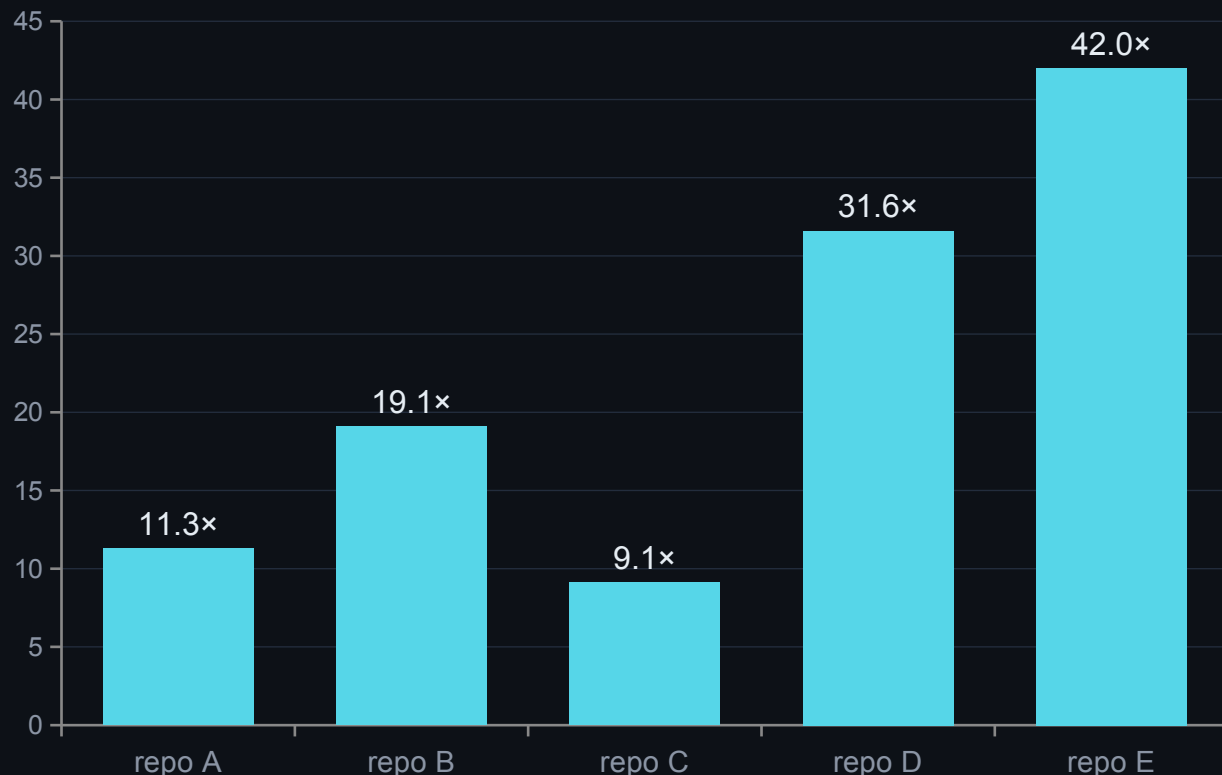
token ceiling for that gain

+3.4%

warm latency for that gain

// compiled, not interpreted

How much faster: same algorithm, 9-42× the speed



Five repositories, tag caches cleared each run, minimum of N runs. Same pipeline on both sides — tree-sitter parse then PageRank — so the gap is compiled versus interpreted.

1,560-file repository

~1 s cold · 0.18 s warm

Aider's map of the same tree: 40 s cold

Where the second goes (self-profiled)

parse ~780 ms → 33 ms warm (24×, the cache)

resolve + PageRank together: ~21 ms

The ranking is the cheap part — the graph math was never the bottleneck.

// the performance sprint

Half the wait: work removed before work optimized

100 ms

cold C++ run — was 190 ms, -47%

20 ms

warm cache run — was 40 ms, half

< 3 ms

graph + rank + emit — below the noise floor

green

determinism, ASan, det-gate and cache gates — unchanged

file ingest

FILE* whole-file reads; CSV counting became a byte scan — iostream out of the hot path

--no-cache

skips contentHash64() entirely when no lookup or save can use it — pure waste removed

capture gating

value-use capture runs only for --uses, --metrics, --for and --exemplar — side captures 404 → 57 ms

parallel parse

query compile overlaps parse scheduling; files schedule by size — ids stay deterministic

backlog cap

parsed-file backlog bounded at 4 files / 8 MiB per worker — memory pressure stays sane

pre-release performance sprint — measured on the private ~1,500-file C++ corpus (historical, not publicly reproducible); same map contract: byte-identical output before and after

// where the time goes now

Faster at scale — and the long pole is the parser

radix edges

numeric keys for graph edges — ~6× edge sort at scale, on the production graph path

radix scores

score/id ordering made numeric — 5.8–8.5× in benchmarks, deterministic tie-break kept

timsort scratch

caller-owned scratch buffer, no per-frame allocation — best on nearly-sorted updates

pdqsort bench

added as the adversarial baseline; std::sort held up well — the baseline keeps us honest

S+tree hot map

bounded-scratch dynamic map: 20.4 ms vs 68.3 ms lookup — a fixed-cap guard reports overflow

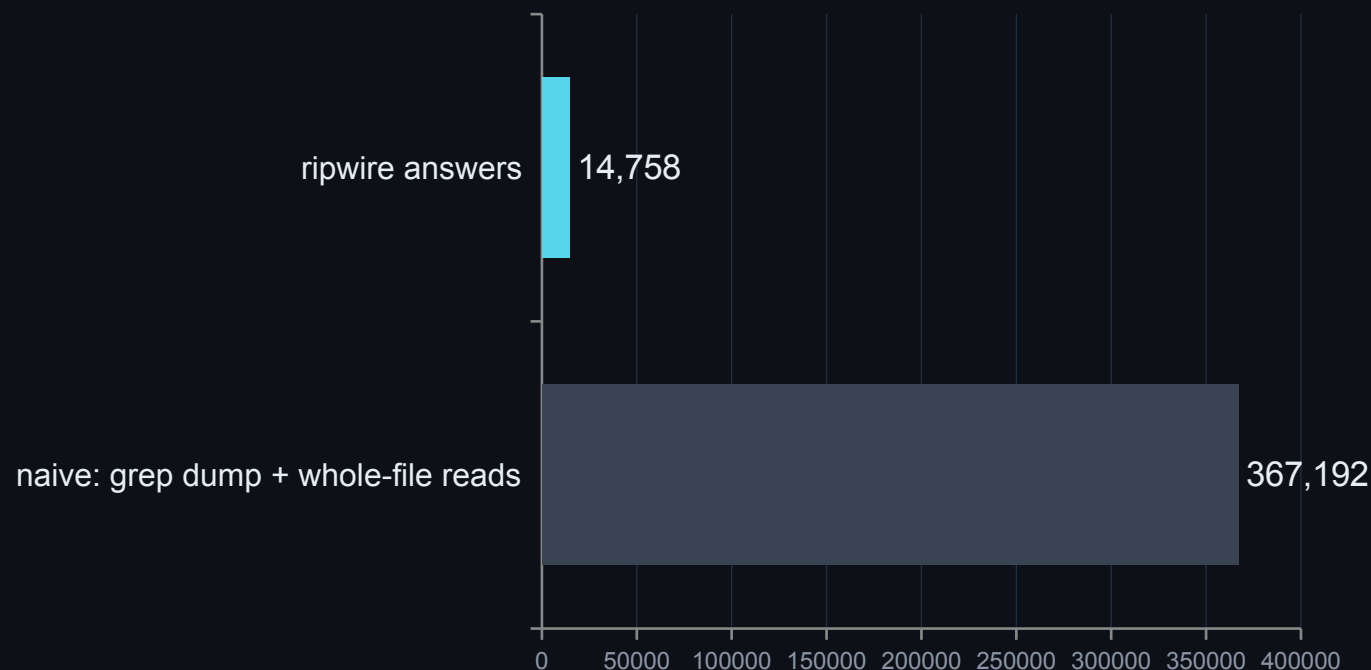
hash lookup

unordered_dense HashMap, reserve() before ingest — no std::unordered_map on any hot path

Cold profiles are now tree-sitter parse + tag-query capture, not graph math. Demand-driven captures already cut 404 → 57 ms; the next wins are query- and grammar-specific. *Same stuff it needs to do — just a lot less waiting.*

```
// the token bill, cut
```

Six real agent questions: 24.9× fewer tokens than the naive read



96.0%

fewer tokens — 14,758 vs 367,192, tiktoken cl100k_base, six questions: who-calls, orientation, blast radius ...

The anti-headline, published next to it: **--grep costs +19.7% tokens (-11.2%)** — not every verb is a reducer, and the docs say which.

Caveat carried with every figure on this slide: measured 2026-06-20 on a large private C++ corpus — historical, private, not publicly reproducible. The public, gated companions are the 67.0% element-byte reduction and LocBench's -39.4% ceiling.

// the cost lever

Smaller answers that are also better answers

67.0%

fewer element bytes at top-50 — the --pack-signatures ladder,
root-neutralised measurement

Ask for a symbol by NAME and the router uses the
name-exact ranker:

MRR 0.859 → 0.993

recall@1 76.7% → 98.7%

name-shaped queries on src/, held-out
labels

This figure survived its own audit: three corrections deep, re-derived
root-neutrally after the original was shown to depend on how the
corpus path was spelled. A gate re-derives it against every fresh
capture.

And the pollution metric — fixture and generated paths
contaminating results — is driven to 0.0% on the ranking lane
while recall went UP, not down.

```
// the half no other tool ships
```

The tripwire: honesty as an output contract

`counts_floor="1"`

call edges come from source text — dynamic dispatch adds no edge, so every count that cannot claim completeness is labelled a floor, in the output itself

`a zero is a measurement`

zero means none found, never none exists — and absent $\neq$ 0

`truncation is disclosed`

every cap, page seam and budget cut is named in the header before you read a row

`refusals teach`

a refused query names the flag, the problem, and a working example — never a bare error

```
$ ripwire . --callers=rankGraphTeleport

<callers of="rankGraphTeleport"
  defs="1" count="6"
  counts_floor="1">
<s t="fn" n="runEval" .../>
<s t="fn" n="rankGraph" .../>
...
</callers>
```

`docs/EVALS.md §8` — the list of numbers this project refuses to publish, each with the reason. The counterexample section is not an afterthought.

// how it stays true

Proven, not promised

315 gate scripts

the suite runs on every push — plus determinism, cache-transparency and golden contracts; the gate count itself is gated against the runner's own loop

byte-identical, always

two runs over the same tree produce the same bytes; warm equals cold. Enforced in CI, twice — Release AND a plain flavour, because NDEBUB once blinded a whole class of checks

differential refactoring

a refactor must prove it changed nothing observable: two binaries, hundreds of argv vectors, stdout + stderr + exit codes byte-identical

held-out labels, authored blind

eval labels were written by reading source before the ranker ever ran on them — so the eval is allowed to say the ranker is wrong. It has.

sanitizer wall

ASan + UBSan + integer + float-cast, no-recover; TSan separately; leak suppression is one pinned file

its own output is audited

the showcase capture is regenerated and adversarially re-read as a claims corpus — the methodology ships in docs/METHODOLOGY.md

// own the losses — they are in the README too

Where it loses, published with the wins

--grep costs more than it saves

+19.7% tokens / -11.2% — measured, in the docs, next to the flag it indicts

PageRank is the wrong co-change ranker

3.8% recall@5 against 40.3% for plain lexical — relatedness is lexical; importance is structural

a signature can outweigh its body

one symbol's signature + doc comment: 303 bytes; its full body: 158 — the compact form lost

strict multi-file localization stays hard

single-file gold 73.4% vs multi-file 18.2% (held-out); the same cliff on every corpus — open headroom, said plainly

A tool that publishes its counterexamples is a tool whose wins you can believe.

```
// built for the agent's seat
```

Wire it into your agent in one command

```
$ ripwire wrap claude
```

(also: cursor, codex, windsurf, gemini, aider)

30 MCP verbs

15 read verbs mirroring the CLI, 12 flagship reflexes (impact, uses, edit_check, from_trace, connect ...), 3 span-addressed edit verbs with a safety contract

lazy-body handles

read verbs return signatures and a stable handle; the agent fetches a body only when it decides it needs one — names by default, bytes on request

17 agent skills

moment-matched skill files (orient, navigate, change-check, quality-bar ...) installed by skills/install.sh

prompts/ — self-improvement

ten copy-paste orchestrator loops: run the same audit, eval and head-to-head machinery that built the tool, on your own repo

the MCP server exposes the same deterministic engine — one index, shared with the CLI, staleness-checked

// standing on giants

The research inside — classic and current

27 repositories + 27 papers folded · 220 more surveyed and labeled — every row with the lesson taken and where it lives: docs/LINEAGE.md

1976

Cyclomatic complexity — McCabe

the metric behind --hotspots

1994

Okapi BM25 — Robertson · Spärck Jones

the lexical ranker (twice: subtoken+body, whole-name)

1998

Personalized PageRank — Page · Brin

the headline importance signal

1999

HITS hubs & authorities — Kleinberg

the alternate lens (--rank-by=authority|hub)

2008

Louvain modularity — Blondel et al.

the module / community map

2009

Reciprocal Rank Fusion — Cormack et al.

deterministic signal fusion (--rank-by=rrf)

TDAD · **arXiv 2603.17973**

a static test-map cut agent-caused regressions 6.08% → 1.82%; prose TDD instructions alone made agents WORSE → the shape of --test-gate

What-to-Retrieve · **arXiv 2503.20589**

retrieval selection beats retrieval volume for coding agents → the selection-over-dumping thesis

LocAgent / Loc-Bench (2025)

the localization metric (strict Acc@k) and the frozen 560-instance dataset every accuracy number here is scored on

tree-sitter

the incremental GLR parsing substrate — 15 grammars vendored, one parser per thread

counts gated in-repo (readmedriftcheck arm E derives them from LINEAGE.md's own tables) — the lineage is the design rationale, not decoration

```
// sixty seconds to the first ranked map
```

Quickstart

```
git clone <repo> && cd ripwire  
cmake -S . -B build && cmake --build build -j      # hermetic: works with the network off  
./build/ripwire --help
```

```
ripwire .
```

the ranked map — start here on any unfamiliar repo

```
ripwire . --for="cache invalidation"
```

the task lens: what to touch, ranked

```
ripwire . --callers=someFunction
```

who calls it — with the honesty marker attached

```
ripwire . --test-gate
```

before you commit: exactly which tests must run

ripgrep for the speed. **tripwire** for the honesty. Apache-2.0.